

**Name: Suriya Priya K**

**Reg No:192521195**

**Course Code: CSA0312**

**Course: Data Structures**



## PROBLEM UNDERSTANDING AND FORMULATION:

### 1.1 Problem Statement:

A smart city disaster management authority needs an intelligent emergency evacuation routing system for a large city containing more than 8,000 intersections and 20,000 weighted roads.

During disasters such as floods, earthquakes, industrial accidents and large-scale fires, roads conditions can change rapidly. Roads may become blocked, damaged or congested, while emergency restrictions may increase travel costs.

Therefore, a conventional static shortest path system is not sufficient.

The proposed system represents:

- Intersections as vertices.
- Roads as weighted edges
- Travel time/cost as the edge weight
- Blocked roads using an active/inactive status.

- Emergency locations as source vertices.
- Evacuation centres as destination vertices.

## 1.2 Expected Outcomes:

The system should:

1. Represent a large road network efficiently.
2. Store more than 8,000 intersections and 20,000 roads.
3. Support dynamic road-weight changes.
4. Disable blocked roads without rebuilding the entire graph.
5. Find shortest routes using Dijkstra's algorithm.
6. Use a Hashset to avoid repeatedly processing intersections.
7. Reconstruct the complete route.
8. Detect invalid and unreachable destinations.

## 1.3 Available information:

The system receives the following information.

| Data              | Description.                              |
|-------------------|-------------------------------------------|
| → Intersection ID | Unique identifier for every intersection. |
| → Road source     | starting intersection.                    |

- Frequent road updates
- Multiple source-destination queries.
- Repeated queries must be answered efficiently
- Memory consumption must remain moderate.
- blocked roads must be avoided.

## 2. APPLICATION OF COURSE KNOWLEDGE:

### 2.1 Graph Abstract Data type

The entire city road network can be represented using a graph:

$$G = (V, E)$$

Where:  $\rightarrow V = \text{set of instructions}$   
 $\rightarrow E = \text{set of roads}.$

For example:

$$V = \{0, 1, 2, 3, 4, 5\}$$

$$E = \{ (0, 1, 4), \\ (0, 2, 2), \\ (1, 2, 1), \\ (1, 3, 5), \\ (2, 3, 8), \\ (2, 4, 10), \\ (3, 4, 2), \\ (3, 5, 6), \\ (4, 5, 3) \}$$

| Data                         | Description                            |
|------------------------------|----------------------------------------|
| → Road destination           | Ending intersection                    |
| → Road weight                | Travel cost/time                       |
| → Road status                | Active or blocked.                     |
| → disaster location          | source of evacuation                   |
| → Evacuation center          | Destination                            |
| → Emergency vehicle location | Current position of emergency vehicle. |
| → Network updates            | changes in road cost/status.           |

#### 1.4 Assumptions.

The following assumptions are made:

1. Every intersection has a unique integer ID.
2. Road weights are non-negative
3. A road is represented as a directed edge.
4. An undirected road can be represented by inserting two directed edges.
5. A blocked road is not considered during route calculation
6. A duplicate road does not create another unnecessary edge. instead, its information is updated.

#### 1.5 constraints

The system has the following constraints:

- more than 8,000 vertices.
- More than 20,000 weighted edges.



## 2.2 Selection of Graph Representation

Two main representations were considered:

### Adjacency Matrix

An adjacency matrix requires:

$$O(V^2) \text{ space.}$$

For 8,000 intersections:

$$\begin{aligned} V^2 &= 8000^2 \\ &= 64,000,000 \end{aligned}$$

If every matrix entry is 4-byte integer:

$$\begin{aligned} 64,000,000 \times 4 \\ &= 256,000,000 \text{ bytes} \\ &= 256 \text{ MB.} \end{aligned}$$

This is large for a system whose network contains only approximately 20,000 roads.

### Adjacency List

An adjacency list requires:

$$O(V + E) \text{ space.}$$

Since:

$$V \approx 8000$$

$$E \approx 2000$$

the adjacency-list representation is much more memory efficient

### Selected Representation

Therefore, the proposed system uses:

Hash Map + Adjacency List.

The Hashmap provides efficient access to a vertex, while the adjacency list stores only roads that actually exist.

### 2.3 Data structures used.

| <u>Data structure</u> | <u>Purpose</u>                      |
|-----------------------|-------------------------------------|
| Graph                 | Represents the city.                |
| Hashmap               | Locates vertices efficiently.       |
| Adjacency List        | Stores roads.                       |
| Hashset               | Tracks processed intersections.     |
| min-heap              | Implements priority queue.          |
| Distance Array        | Stores shortest distances, $\infty$ |

### 3. USE A HASHSET TO TRACK PROCESSED INTERSECTIONS

Initially the hashset is empty

processed =  $\{\}$

Processed =  $\{A, B, C\}$ .

parent [E] = C

parent [F] = E

Starting from the destination F,

$F \rightarrow E \rightarrow C \rightarrow A$

The sequence is reversed.

$A \rightarrow C \rightarrow E \rightarrow F$

### Pseudocode:

1. Start from destination.
2. Store the current vertex in the route.
3. Move to its parent vertex.
4. Reverse the stored route.
5. Display the final route.

Ex: For the graph.

$A \rightarrow C \rightarrow E \rightarrow F$

weights: 3, 6, 2 (start from F)

$A \rightarrow C = 3$

$C \rightarrow E = 6$

$E \rightarrow F = 2$

$$\therefore \text{Total cost} = 3 + 6 + 2 = 11$$

Shortest route:  $A \rightarrow C \rightarrow E \rightarrow F$ .



If vertex is hashset

→ skip the vertex.

else

→ process the vertex

→ Add it to hash set

Eg:  $A \rightarrow C \rightarrow B \rightarrow D$

Then:  $\text{hashset} = \{A, C, B, D\}$

If the algorithm C again, it checks the hashset and skip C.

advantages:

- 1) Avoids repeated processing
- 2) Improves routing efficiency.
- 3) help manage large number of intersections.
- 4) supports faster execution of dijkstra's algorithm.

#### 4. Implement route reconstruction:

After Dijkstra's algorithm finds the minimum total cost, the system must identify the actual sequence of intersection.

Ex:  $A \rightarrow C \rightarrow E \rightarrow F$

$\text{Parent}[C] = A$

The routing systems must be able to handle abnormal situations because disaster conditions can change the road network.

blocked road:

$C \xrightarrow{\text{X}} D$   
blocked.

Input

Invalid vertex:

Input: vertex = 9001

Output: Invalid intersection.

unreachable destination

Output

Evacuation center is unreachable.

Time and space complexity

Let

$V$  = no. of intersections

$E$  = no. of roads.

Total travel cost = 11

## 7. Implement route caching for repeated queries

When a route request is received, the system checks the cache.

If route exists in cache

→ return stored route.

Else

→ Run Dijkstra

→ store the new route

→ Return the route.

Ex: Request :

Source = A

Destination = F

Cache

(A, F) → A → C → E → F

cost = 11

⇒ Note: If the road weight or road status changes, the affected cached route should be updated or removed. Otherwise, the system may return an outdated route.

handling blocked roads, invalid vertices, duplicate roads and unreachable destination.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <limits.h>
```

```
#define MAX 100
```

```
#define HEAP_MAX 1000
```

```
#define INF INT_MAX
```

```
/* =====
```

```
EDGE STRUCTURE
```

```
===== */
```

```
typedef struct Edge {
```

```
    int to;
```

```
    int weight;
```

```
    int active;
```

```
    struct Edge *next;
```

```
} Edge;
```

```
/* =====
```

```
GRAPH
```

```
===== */
```

```
Edge *graph[MAX] = {NULL};
```

```
/* =====
```

```
MIN HEAP
```

```
===== */
```

```
typedef struct {
```

```
    int vertex;
```

```
    int distance;
```

```
} HeapNode;
```

```
HeapNode heap[HEAP_MAX];
```

```
int heapSize = 0;
```

```
/* =====
```

```
VISITED SET
```

```
===== */
```

```
int visited[MAX];
```

```
/* =====
```

```
HEAP FUNCTIONS
```

```
===== */
```

```
void swap(HeapNode *a, HeapNode *b)
```

```
{
```

```
    HeapNode temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int heapPush(int vertex, int distance)
```

```
{
```

```
    if (heapSize >= HEAP_MAX)
```

```
        return 0;
```

```
    int j = heapSize++;
```

```
    heap[j].vertex = vertex;
```

```
    heap[j].distance = distance;
```

```
    while (j > 0)
```



```

{
    int parent = (i - 1) / 2;

    if (heap[parent].distance <= heap[i].distance)
        break;

    swap(&heap[parent], &heap[i]);
    i = parent;
}

return 1;
}

```

```

HeapNode heapPop()
{
    HeapNode result;

    result.vertex = -1;
    result.distance = INF;

    if (heapSize == 0)
        return result;

    result = heap[0];

```

```
heapSize--;
```

```
if (heapSize > 0)
```

```
{
```

```
    heap[0] = heap[heapSize];
```

```
    int j = 0;
```

```
    while (1)
```

```
    {
```

```
        int left = 2 * j + 1;
```

```
        int right = 2 * j + 2;
```

```
        int smallest = j;
```

```
        if (left < heapSize &&
```

```
            heap[left].distance < heap[smallest].distance)
```

```
        {
```

```
            smallest = left;
```

```
        }
```

```
        if (right < heapSize &&
```

```
            heap[right].distance < heap[smallest].distance)
```

```
        {
```

```
}
```

```
/* =====
```

```
TEST 4
```

```
===== */
```

```
printf("\nTEST 4: UNREACHABLE DESTINATION\n");
```

```
blockRoad(0, 1);
```

```
blockRoad(0, 2);
```

```
if (!dijkstra(vertices, 0, 5, distance, parent))
```

```
{
```

```
    printf("\nDestination 5 is UNREACHABLE from 0.\n");
```

```
}
```

```
/* =====
```

```
CLEANUP
```

```
===== */
```

```
freeGraph(vertices);
```

```
printf("\nProgram completed successfully.\n");
```

```
/* =====  
    FIND ROAD  
    ===== */
```

```
Edge *findRoad(int from, int to)
```

```
{  
    if (from < 0 || from >= MAX)  
        return NULL;
```

```
    Edge *current = graph[from];
```

```
    while (current != NULL)
```

```
{  
    if (current->to == to)  
        return current;
```

```
    current = current->next;
```

```
}
```

```
    return NULL;
```

```
}
```

```
/* =====  
    BLOCK ROAD
```

```

{
    if (from < 0 || from >= MAX ||
        to < 0 || to >= MAX ||
        weight < 0)
    {
        return;
    }

    Edge *newEdge = (Edge *)malloc(sizeof(Edge));

    if (newEdge == NULL)
    {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }

    newEdge->to = to;
    newEdge->weight = weight;
    newEdge->active = 1;
    newEdge->next = graph[from];

    graph[from] = newEdge;
}

```

```
        smallest = right;
    }

    if (smallest == j)
        break;

    swap(&heap[j], &heap[smallest]);
    j = smallest;
}
}
```

```
return result;
}
```

```
int heapEmpty()
{
    return heapSize == 0;
}
```

```
/* =====
ADD ROAD
===== */
```

```
void addRoad(int from, int to, int weight)
```

## TEST 1: NORMAL ROUTE

Shortest Route: 0 -> 1 -> 3 -> 4 -> 5

Minimum Cost: 14

## TEST 2: ROAD BLOCKAGE

Road 3 -> 4 BLOCKED

Shortest Route: 0 -> 1 -> 3 -> 5

Minimum Cost: 15

## INTELLIGENT EMERGENCY EVACUATION SYSTEM

### ===== ROAD NETWORK =====

0: 2(cost=2,OPEN) -> 1(cost=4,OPEN)

1: 3(cost=5,OPEN) -> 2(cost=1,OPEN)

2: 4(cost=10,OPEN) -> 3(cost=8,OPEN)

3: 5(cost=6,OPEN) -> 4(cost=2,OPEN)

4: 5(cost=3,OPEN)

5: No roads

=====



### TEST 3: DYNAMIC ROAD UPDATE

Road 2 -> 4 updated to cost 3

Shortest Route: 0 -> 2 -> 4 -> 5

Minimum Cost: 8

### TEST 4: UNREACHABLE DESTINATION

Road 0 -> 1 BLOCKED

Road 0 -> 2 BLOCKED

Destination 5 is UNREACHABLE from 0.

Program completed successfully.